Repaso: Programación Orientada a Objetos

Antes de comenzar el curso

Índice

1. ¿Para quién es este documento?
2. Encapsulamiento
3. Herencia
4. Abstracción
5. Polimorfismo
6. Lo que debes traer listo

¿Para quién es este documento?

Este documento es para ti si tomaste este curso y sientes que algunos conceptos de la sección uno te resultaron confusos o poco familiares.

No es un bloqueo — es un trampolín.

Si lees este PDF y los ejemplos te resultan naturales, estás listo para aprovechar el curso al máximo. Si encuentras conceptos que no recuerdas bien, es una señal de que vale la pena repasar programación orientada a objetos antes de continuar.

> *Este curso asume que ya conoces los cuatro pilares de la programación orientada a objetos. No los enseña desde cero — los usa como base para construir conceptos más avanzados como interfaces, genéricos y expresiones lambda.*

Encapsulamiento

¿Qué es?

Encapsulamiento es **proteger el estado interno de un objeto** para que nadie pueda modificarlo directamente desde afuera. El objeto controla cómo se accede y cómo se modifica su propia información.

¿Por qué importa?

Sin encapsulamiento cualquiera puede poner un valor inválido en tu objeto y romper su comportamiento. Con encapsulamiento tú decides qué se puede modificar y bajo qué condiciones.

Ejemplo resuelto

```java
public class Car {

    // los campos son privados — nadie los toca directamente
    private String brand;
    private int speed;

    public Car(String brand) {
        this.brand = brand;
        this.speed = 0; // un auto siempre arranca en cero
    }

    // getter — solo lectura
    public String getBrand() {
        return brand;
    }

    // getter
    public int getSpeed() {
        return speed;
    }

    // setter con validación — el objeto controla sus propias reglas
    public void accelerate(int amount) {
        if (amount > 0) {
            this.speed += amount;  // solo acepta valores positivos
        }
    }

    public void brake(int amount) {
        if (amount > 0 && this.speed > 0) {
            this.speed = Math.max(0, this.speed - amount); // no puede ser
negativa
        }
    }
}
```

```java
public class Main {
    public static void main(String[] args) {

        Car car = new Car("Toyota");

        car.accelerate(60);
        IO.println(car.getSpeed()); // 60

        car.brake(20);
        IO.println(car.getSpeed()); // 40

        // car.speed = -100; — esto NO compila
        // nadie puede romper el estado del objeto desde afuera
    }
}
```

Lo que debes recordar

- Los campos siempre van `private`
- El acceso se controla con getters y setters
- El setter puede validar antes de asignar — el objeto protege su propio estado

---

Herencia

¿Qué es?

Herencia es la capacidad de crear una clase nueva **basada en una clase existente**, reutilizando su comportamiento y agregando o modificando lo que sea necesario. La clase nueva se llama subclase y la original se llama superclase.

¿Por qué importa?

Evita repetir código. Si tienes comportamiento común entre varios tipos de objetos, lo defines una sola vez en la superclase y todas las subclases lo heredan automáticamente.

Ejemplo resuelto

```java
// superclase — comportamiento común para todos los vehículos de motor
public class Car {

    private String brand;
    private int speed;

    public Car(String brand) {
        this.brand = brand;
        this.speed = 0;
    }

    public String getBrand() { return brand; }
    public int getSpeed()    { return speed; }

    public void accelerate(int amount) {
        if (amount > 0) this.speed += amount;
    }

    public String getStatus() {
        return brand + " going at " + speed + " km/h";
    }
}
```

```java
// subclase — hereda todo de Car y agrega lo propio de un auto eléctrico
public class ElectricCar extends Car {

    private int batteryLevel; // campo propio — no existe en Car

    public ElectricCar(String brand, int batteryLevel) {
        super(brand);                    // llama al constructor de Car
        this.batteryLevel = batteryLevel;
    }

    public int getBatteryLevel() { return batteryLevel; }

    // método propio — solo existe en ElectricCar
    public void charge(int amount) {
        this.batteryLevel = Math.min(100, this.batteryLevel + amount);
    }

    // @Override — reemplaza el comportamiento del padre
    @Override
    public String getStatus() {
        return super.getStatus() + " | Battery: " + batteryLevel + "%";
    }
}
```

```java
public class Main {
    public static void main(String[] args) {

        ElectricCar tesla = new ElectricCar("Tesla", 80);

        tesla.accelerate(120);  // heredado de Car
        tesla.charge(15);       // propio de ElectricCar

        IO.println(tesla.getStatus());
        // Tesla going at 120 km/h | Battery: 95%
    }
}
```

Lo que debes recordar

- `extends` conecta la subclase con la superclase
- `super(...)` llama al constructor del padre — siempre va primero
- `@Override` indica que estás reemplazando un método del padre
- La subclase hereda todo lo `public` y `protected` del padre

Abstracción

¿Qué es?

Abstracción es **definir un contrato** — qué debe hacer algo — sin decir cómo lo hace. Una clase abstracta o una interfaz define las operaciones obligatorias que todas sus implementaciones deben cumplir.

¿Por qué importa?

Cuando defines una abstracción, el código que la usa no necesita saber los detalles de implementación. Solo sabe qué puede pedirle al objeto — y eso es suficiente para trabajar con él.

Ejemplo resuelto

```java
// clase abstracta — define el contrato para todos los vehículos
public abstract class Vehicle {

    private String brand;

    public Vehicle(String brand) {
        this.brand = brand;
    }

    public String getBrand() { return brand; }

    // método abstracto — todas las subclases DEBEN implementarlo
    // no tiene cuerpo — solo define QUÉ debe hacerse, no CÓMO
    public abstract void move();

    public abstract String getFuelType();

    // método concreto — comportamiento común que no cambia
    public void printInfo() {
        IO.println(brand + " uses " + getFuelType() + " as fuel");
    }
}
```

```java
// implementación 1 — define el CÓMO para un auto de gasolina
public class GasCar extends Vehicle {

    public GasCar(String brand) {
        super(brand);
    }

    @Override
    public void move() {
        IO.println(getBrand() + " moves burning gasoline");
    }

    @Override
    public String getFuelType() {
        return "gasoline";
    }
}
```

```java
// implementación 2 — define el CÓMO para un auto eléctrico
public class ElectricCar extends Vehicle {

    public ElectricCar(String brand) {
        super(brand);
    }

    @Override
    public void move() {
        IO.println(getBrand() + " moves using electric power");
    }

    @Override
    public String getFuelType() {
        return "electricity";
    }
}
```

```java
public class Main {
    public static void main(String[] args) {

        // Vehicle v = new Vehicle("X"); — NO compila, es abstracta
        // solo puedes instanciar las implementaciones concretas

        GasCar gas      = new GasCar("Ford");
        ElectricCar ev  = new ElectricCar("Tesla");

        gas.printInfo();   // Ford uses gasoline as fuel
        ev.printInfo();    // Tesla uses electricity as fuel

        gas.move();        // Ford moves burning gasoline
        ev.move();         // Tesla moves using electric power
    }
}
```

Lo que debes recordar

- Una clase `abstract` no se puede instanciar directamente
- Los métodos `abstract` no tienen cuerpo — solo la firma
- Toda subclase concreta **debe** implementar todos los métodos abstractos
- Los métodos no abstractos en la clase abstracta son comportamiento compartido

---

Polimorfismo

¿Qué es?

Polimorfismo significa que **un mismo tipo puede tomar muchas formas.** Un objeto de tipo `Vehicle` puede ser en realidad un `GasCar`, un `ElectricCar` o cualquier otra subclase — y cada uno responde a los mismos mensajes de forma diferente.

¿Por qué importa?

Es el pilar que hace que el código sea flexible y extensible. Puedes agregar nuevos tipos de vehículos sin cambiar el código que los usa — solo cumpliendo el contrato.

Ejemplo resuelto

```java
public class Main {
    public static void main(String[] args) {

        // todos son Vehicle — pero cada uno es distinto por dentro
        List<Vehicle> fleet = new ArrayList<>();
        fleet.add(new GasCar("Ford"));
        fleet.add(new ElectricCar("Tesla"));
        fleet.add(new GasCar("Toyota"));
        fleet.add(new ElectricCar("Rivian"));

        // el mismo mensaje move() — cuatro comportamientos distintos
        for (Vehicle vehicle : fleet) {
            vehicle.move(); // Java sabe cuál implementación llamar en
tiempo de ejecución
        }

        // Ford moves burning gasoline
        // Tesla moves using electric power
        // Toyota moves burning gasoline
        // Rivian moves using electric power
    }
}
```

```java
// el método no sabe qué tipo exacto recibe — solo sabe que es un Vehicle
// eso es polimorfismo: el código trabaja con la abstracción, no con el
detalle
public static void startVehicle(Vehicle vehicle) {
    IO.println("Starting: " + vehicle.getBrand());
    vehicle.move();
}
```

```java
public class Main {
    public static void main(String[] args) {

        // puedes pasar cualquier subclase — el método no cambia
        startVehicle(new GasCar("Honda"));      // Starting: Honda
        startVehicle(new ElectricCar("BMW"));   // Starting: BMW
    }
}
```

Lo que debes recordar

- Una variable del tipo padre puede apuntar a cualquier subclase
- En tiempo de ejecución Java decide qué implementación llamar
- El código que usa la abstracción no necesita cambiar cuando agregas nuevos tipos
- Polimorfismo + abstracción = código flexible y extensible

Lo que debes traer listo

Si leíste este documento y los ejemplos te resultaron claros, estás listo para aprovechar el curso al máximo.

Antes de continuar, verifica que puedes responder estas preguntas sin dudar:

Concepto	Pregunta de verificación
Encapsulamiento	¿Por qué los campos van `private` y no `public`?
Herencia	¿Para qué sirve `super()` en el constructor?
Abstracción	¿Por qué no puedes instanciar una clase `abstract`?
Polimorfismo	¿Qué pasa cuando llamas `move()` sobre una variable de tipo `Vehicle`?

> *Si alguna de estas preguntas te genera duda, te recomendamos repasar programación orientada a objetos antes de continuar. El curso construye directamente sobre estos conceptos desde la primera sección.*

© Rene Calderón